
Documento Tecnico Architettuale

LNI MESSINA — WEB APP PRE-ADESIONE E CHECK-IN

DOCUMENTO TECNICO ARCHITETTURALE COMPLETO

Versione 3.2 — Anti-bot, QR activation fix, GDPR update

1. INTRODUZIONE

Obiettivo del progetto

Realizzare una nuova WEB APP moderna, performante, sicura e facilmente mantenibile per la gestione:

- delle richieste di pre-adesione;
- dei soci attivi;
- del check-in tramite QRCode;
- del controllo accessi;
- del logging accessi;
- del recupero QRCode;
- dell'amministrazione utenti;
- della compliance GDPR.

La nuova architettura dovrà sostituire integralmente l'attuale implementazione basata su:

- Google Apps Script;
- Google Sheets;
- TinyURL;
- codici manuali testuali.

2. OBIETTIVI DELLA MIGRAZIONE

Obiettivi funzionali

La nuova soluzione deve:

- mantenere l'attuale workflow operativo;
- migliorare velocità e stabilità;
- eliminare dipendenze da Google Apps Script;
- migliorare sicurezza e controllo accessi;
- introdurre QRCode dinamici;
- introdurre scanner QR tramite smartphone;
- introdurre ruoli amministrativi;
- migliorare audit e logging;
- migliorare UX mobile;
- mantenere semplicità operativa.

3. PRINCIPI ARCHITETTURALI

Principi fondamentali

Il sistema deve essere:

- mobile-first;
- semplice da usare;
- veloce;
- sicuro;
- facilmente scalabile;
- GDPR compliant;
- modulare;
- facilmente estendibile.

4. ARCHITETTURA TECNOLOGICA

4.1 FRONTEND

Framework

Utilizzare:

Next.js

Versione consigliata:

- Next.js App Router.

UI Framework

Utilizzare:

TailwindCSS

UI Philosophy

L'interfaccia deve essere:

- molto semplice;
- adatta a operatori non tecnici;
- ottimizzata per smartphone;
- con:
- pulsanti grandi;
- font grandi;
- interazioni touch;
- layout pulito.

NON creare dashboard enterprise complesse.

PWA

La WEB APP deve essere configurata come:

PWA - Progressive Web App

Funzioni richieste:

- installabile su smartphone;
- fullscreen;
- icona home;
- supporto camera;
- UX quasi nativa.

4.2 BACKEND

Backend principale

Utilizzare:

Supabase

Servizi Supabase utilizzati

PostgreSQL

Database principale.

Auth

Gestione autenticazione admin/operatori.

Storage

Archiviazione QRCode.

Edge Functions

Logica backend avanzata.

Row Level Security

Protezione dati.

Cron Jobs

Cleanup GDPR automatico.

4.3 REPOSITORY

Versioning

Utilizzare:

GitHub

Repository structure

Documentazione/ — Documentazione tecnica e guide

src/

app/ — Route Next.js (App Router)

admin/ — Pagine amministrazione
api/ — API route
checkin/ — Scanner check-in
login/ — Login amministratore
operator/ — Pagine operatore
stampa/ — Pagine stampabili
validate/ — Validazione QR
register/ — Pre-iscrizione
recover-qr/ — Recupero QR
components/ — Componenti riutilizzabili
lib/ — Librerie e utility
supabase/ — Schema DB e migrazioni
public/ — Asset statici (logo, QR code PNG)

4.4 HOSTING

Frontend hosting

Utilizzare:

Vercel Free Plan

Dominio di produzione:

<https://registrazionesoci.vercel.app>

5. MODELLO OPERATIVO

5.1 TIPOLOGIE UTENTI

Il sistema gestisce due categorie principali.

PRE_MEMBER

Utente che:

- compila il form pubblico;
- richiede pre-adesione;
- riceve QRCode temporaneo.

Validità:

- 90 giorni.

ACTIVE_MEMBER

Socio attivo:

- inserito manualmente da amministratore;
- già riconosciuto dalla LNI.

Può:

- ricevere QRCode;
- recuperare QRCode;
- effettuare check-in.

5.2 RUOLI OPERATIVI

CHECKIN_OPERATOR

Può:

- accedere al registro accessi (login operatore);
- consultare esiti check-in con filtro orario.

NON può:

- modificare utenti;
- vedere dati completi;
- esportare dati.

Nota: lo scanner QR in `/checkin` è pubblico e non richiede login.

L'autenticazione operatore è necessaria solo per il registro accessi (`/operator/accessi`).

ADMIN_MONITOR

Può:

- monitorare utenti;
- visualizzare statistiche;
- visualizzare log;
- visualizzare check-in.

NON può:

- modificare utenti;
- cancellare dati;
- inviare QR.

ADMIN_FULL

Può:

- gestire utenti;
- creare soci attivi;
- modificare dati;
- rigenerare QR;
- revocare QR;
- inviare QR;
- esportare dati;
- gestire sistema.

6. USER EXPERIENCE

6.1 MODALITÀ PUBBLICA

Route pubbliche

/

Homepage.

/register

Registrazione pre-adesione.

/recover-qr

Recupero QRCode.

/checkin

Scanner QR code per check-in. Accessibile a tutti, nessun login richiesto.

/validate/[token]

Validazione QR code personale.

/stampa/registrazione

Pagina ottimizzata per stampa con QR code pre-iscrizione.

6.2 MODALITÀ OPERATORE

Route

/operator

Home operatore (richiede login). Hub con pulsanti per scanner check-in e registro accessi.

Credenziali: utente `operatore`, password `verifica1!`.

/operator/accessi

Registro accessi con filtro 24h/12h/6h/4h (richiede login). Mostra esiti check-in con badge colore.

/operator/checkins

(Alternativa) Monitoraggio check-in via API route con Bearer token.

Login operatore

L'autenticazione operatore usa Supabase Auth con email `operatore@leganavale.it`.

La UI presenta un form semplificato (solo utente/password, senza mostrare l'email).

Il componente `OperatorGuard.tsx` gestisce login/logout e verifica sessione esistente all'avvio.

6.3 MODALITÀ CHECK-IN (SCANNER)

Route

/checkin

Interfaccia fullscreen scanner (pubblica, nessun login).

UX scanner

L'interfaccia scanner deve:

- aprire camera automaticamente;
- essere fullscreen;
- avere:
- feedback verde/rosso;
- vibrazione smartphone;
- suono conferma;
- esito immediato.

6.4 MODALITÀ ADMIN

Route

/login

Login amministratore con email/password Supabase Auth.

/admin

Dashboard.

/admin/users

Gestione utenti.

/admin/members

Gestione soci attivi.

/admin/checkins

Monitoraggio accessi.

/admin/logs

Audit logs.

7. DATABASE DESIGN

7.1 TABELLA USERS

```
create table users (  
  id uuid primary key default gen_random_uuid(),  
  
  first_name text not null,  
  last_name text not null,  
  
  email text not null,  
  phone text,  
  
  user_type text not null,
```

-- pre_member	active_member
---------------	---------------

```
status text not null,
```

-- active	expired	revoked
-----------	---------	---------

```
gdpr_consent boolean default false,  
marketing_consent boolean default false,  
registration_date timestamptz default now(),  
expiration_date timestamptz,  
created_by uuid,  
created_at timestamptz default now(),  
updated_at timestamptz default now()  
);
```

7.2 TABELLA QR_TOKENS

```
create table qr_tokens (  
  id uuid primary key default gen_random_uuid(),  
  
  user_id uuid references users(id),
```

```
token text unique not null,  
  
is_active boolean default true,  
  
created_at timestamptz default now(),  
revoked_at timestamptz  
  
);  
  
---
```

7.3 TABELLA CHECKIN_LOGS

```
create table checkin_logs (  
id uuid primary key default gen_random_uuid(),  
  
user_id uuid references users(id),  
  
operator_id uuid,  
  
checkin_result text,  
  
device_info text,  
  
ip_address text,  
  
created_at timestamptz default now()  
  
);  
  
---
```

7.4 TABELLA ADMIN_USERS

```
create table admin_users (  
id uuid primary key default gen_random_uuid(),  
  
email text unique not null,  
  
role text not null,  
  
created_at timestamptz default now()  
  
);  
  
---
```

7.5 TABELLA AUDIT_LOGS

```
create table audit_logs (  
id uuid primary key default gen_random_uuid(),  
  
actor_id uuid,  
  
action_type text,
```

```
target_type text,  
target_id text,  
metadata jsonb,  
created_at timestamptz default now()  
);
```

7.6 TABELLA HOUSEHOLDS (OPZIONALE MA CONSIGLIATA)

Necessaria per gestione nuclei familiari.

```
create table households (  
id uuid primary key default gen_random_uuid(),  
email text,  
phone text,  
created_at timestamptz default now()  
);
```

8. LOGICA QRCODE

8.1 REQUISITI

Il QRCode NON deve contenere:

- nome;
- cognome;
- email;
- telefono;
- dati personali.

8.2 FORMATO QR

Formato consigliato:

<https://app-domain/validate/TOKEN>

Esempio:

<https://registrazionesoci.vercel.app/validate/8f2d91fa>

8.3 CARATTERISTICHE TOKEN

Il token deve essere:

- randomico;
- univoco;
- rigenerabile;
- revocabile.

8.4 QRCODE RECOVERY

Il sistema deve consentire:

- reinvio QR;
- rigenerazione QR;
- invalidazione vecchi QR.

9. REGISTRAZIONE PRE-ADESIONE

9.1 FLUSSO DOUBLE OPT-IN (GDPR COMPLIANT)

PASSO 1

Utente apre `/register` e compila:

- nome, cognome, email, telefono;
- consenso GDPR (obbligatorio);
- consenso marketing (opzionale).

PASSO 2

Sistema:

- valida dati e controlla duplicati;
- crea utente;
- ****logga i consensi**** su `checkin\_logs`:
- `checkin\_result`: `GDPR\_CONSENT` / `MARKETING\_CONSENT`;

- `device\_info`: versione privacy + metodo acquisizione + user agent;
- `ip\_address`: IP del richiedente;
- genera ****confirmation token**** in `qr\_tokens` con `is\_active = false`;
- NON genera ancora il QR code definitivo.

PASSO 3

Sistema invia email di **conferma** con link:

/confirm-email?token=CONFIRM_TOKEN

L'email contiene un pulsante "Conferma Email" e il link testuale.

PASSO 4

Utente clicca il link di conferma.

Sistema:

- trova `qr\_tokens` con `is\_active = false` e token corrispondente (SELECT);
- ****INSERT**** un NUOVO token attivo in `qr\_tokens` (l'UPDATE non è permesso dall'anon key via RLS);
- il vecchio token inattivo rimane come record storico;
- logga evento `EMAIL\_VERIFIED` su `checkin\_logs`;
- mostra il nuovo QR code a schermo;
- invia email con il nuovo QR code allegato.

> **Nota tecnica:** L'UPDATE su `qr\_tokens` è bloccato da RLS per l'anon key.

> Il flusso usa INSERT di un nuovo token attivo invece di UPDATE.

> Schema: `INSERT qr\_tokens(user\_id, token=NUOVO\_TOKEN, is\_active=true)`

PASSO 5

Utente salva o stampa il QR code per il check-in in sede.

9.2 CONSENT LOGGING (GDPR)

Il sistema registra ogni consenso GDPR per garantire compliance.

Dati registrati per ogni consenso (su `checkin\_logs`):

- `user\_id`: riferimento all'utente;
- `checkin\_result`: tipo di consenso (`GDPR\_CONSENT`, `MARKETING\_CONSENT`);
- `device\_info`: versione informativa privacy (`v1\_2024`) + metodo (`web\_form`) + user agent;
- `ip\_address`: indirizzo IP del richiedente;
- `created\_at`: timestamp automatico.

Flusso:

- Al momento della pre-iscrizione, ogni consenso viene registrato come record separato;
- La registrazione avviene via API route lato server (`/api/register`), che estrae l'IP dalla richiesta;
- I dati di logging sono accessibili solo a utenti con ruolo `admin\_full`;

Informativa privacy vigente:

https://www.leganavale.it/mod/aalborg_theme/pages/generic.php?filename=0746730001778785218_InformativaTrattamentoDatiPersonaliRegistrazioneTelematica-Maggio.pdf

9.4 ANTI-BOT MEASURES

Il sistema implementa due livelli di protezione contro attacchi automatici (bot, spam, denial-of-service), sia sul form di registrazione che su quello di recupero QR.

9.4.1 Honeypot Invisibile

Un campo input `website` è posizionato fuori schermo tramite CSS (`position: absolute; left: -9999px`) e nascosto agli screen reader (`aria-hidden="true"`).

- Gli utenti umani NON vedono né compilano questo campo
- I bot tendono a compilare tutti i campi input
- Il server controlla: `if (website) { return 403 }`

Implementazione client: `register/page.tsx`, `recover-qr/page.tsx`

Implementazione server: `api/register/route.ts`, `api/send-qr/route.ts`

9.4.2 Time-to-Submit

Al caricamento della pagina, il componente registra `Date.now()` in un `useRef`. Al submit calcola `Date.now() - formLoadedAt` e invia il valore `elapsed` (millisecondi).

Il server verifica:

- `typeof elapsed === 'number'` (se non inviato -> sospetto)
- `elapsed >= 3000` (3 secondi minimo per un umano per compilare i campi)

Implementazione client: `register/page.tsx`, `recover-qr/page.tsx`

Implementazione server: `api/register/route.ts` (check obbligatorio), `api/send-qr/route.ts` (check condizionale)

9.4.3 Flussi esclusi

Le chiamate a `api/send-qr` provenienti da conferma email (`confirm-email`) NON inviano `elapsed`/`website` perche' partono da una sessione gia' verificata via email.

9.5 CONTROLLI DUPLICATI

Verificare:

- nome;
- cognome;
- email;
- telefono.

9.3 VALIDITÀ

La pre-adesione:

- dura 90 giorni;
- poi viene:
- cancellata;
- oppure archiviata solo per marketing se consentito.

10. GESTIONE SOCI ATTIVI

10.1 INSERIMENTO MANUALE

ADMIN_FULL deve poter:

- inserire socio;
- associare email;
- associare telefono;
- generare QR.

10.2 INVIO QR CODE SU RICHIESTA

Funzione richiesta:

- inserimento email;
- ricerca socio;
- reinvio QRCode.

Valido per:

- soci attivi;
- pre-aderenti.

11. CHECK-IN

11.1 FLUSSO

STEP 1

Operatore apre:

/checkin

STEP 2

Sistema:

- attiva camera;
- attiva scanner QR.

STEP 3

Scanner legge token.

STEP 4

Sistema:

- valida token;
- verifica stato;
- verifica scadenza;
- registra log.

STEP 5

Sistema mostra:

Esito positivo

- nome;
- cognome;
- stato valido.

Esito negativo

- QR non valido;
- QR scaduto;
- QR revocato.

11.2 LIBRERIE QR SCANNER

Utilizzare:

html5-qrcode

oppure:

zxing-js

12. RECOVERY QR CODE

12.1 ROUTE

/recover-qr

12.2 PROCEDURA

Utente inserisce:

- email.

12.3 SICUREZZA

Il sistema NON deve:

- reinviare immediatamente QR.

Il sistema deve:

- inviare link temporaneo;
- valido 15 minuti;
- monouso.

12.4 PROTEZIONI

Implementare:

- rate limiting;
- anti spam;
- anti enumeration.

13. PANNELLO ADMIN

13.1 DASHBOARD

Visualizzare:

- utenti registrati;
- soci attivi;
- check-in giornalieri;
- QR attivi;
- QR revocati;
- utenti scaduti.

13.2 GESTIONE UTENTI

Funzioni:

- ricerca;
- filtro;
- modifica;
- disattivazione;
- esportazione CSV.

13.3 GESTIONE QRCODE

Funzioni:

- rigenerazione;
- revoca;
- reinvio.

13.4 GESTIONE SOCI

Funzioni:

- inserimento manuale;
- modifica;
- attivazione/disattivazione.

13.5 AUDIT LOGS

Visualizzare:

- accessi admin;
- modifiche utenti;
- operazioni sensibili.

14. SICUREZZA

14.1 HTTPS

HTTPS obbligatorio.

14.2 AUTH

Utilizzare:

- Supabase Auth.

14.3 RLS

Abilitare:

- Row Level Security su tutte le tabelle.

14.4 RATE LIMITING

Proteggere:

- login;
- recovery QR;
- scanner;
- API pubbliche.

14.5 LOGGING

Registrare:

- IP;
- device;
- timestamp;
- operazioni admin.

15. GDPR

15.1 CONSERVAZIONE DATI

Pre-aderenti:

- eliminazione automatica dopo 90 giorni.

15.2 CONSENSO MARKETING

Conservare solo:

- utenti con consenso esplicito.

15.3 MINIMIZZAZIONE DATI

Operatori check-in devono vedere SOLO:

- nome;
- cognome;
- stato validità.

15.4 AUDIT

Tracciare:

- accessi admin;
- modifiche;
- esportazioni.

16. EMAIL SYSTEM

16.1 PROVIDER

Utilizzare:

- Resend
- oppure
- Brevo.

16.2 EMAIL PRINCIPALI

Registrazione

Invio QRCode.

Recovery QR

Invio link temporaneo.

Reinvio admin

Invio QR manuale.

17. LIBRERIE CONSIGLIATE

Frontend

Next.js

TailwindCSS

React Hook Form

Zod

QR Scanner

html5-qrcode

QR Generation

qrcode

Backend

Supabase

18. DEPLOYMENT

18.1 WORKFLOW

VSCode

↓

GitHub Push

↓

Vercel Auto Deploy

↓

Production Update

18.2 ENV VARIABLES

Gestire:

- Supabase URL;
- Supabase Key;
- Email API Keys.

NON salvare segreti nel repository.

19. STRUTTURA DOCUMENTAZIONE

Creare:

/docs/architecture.md

/docs/database.md

/docs/api.md

/docs/security.md

/docs/deployment.md

/docs/gdpr.md

20. PRIORITÀ IMPLEMENTAZIONE

FASE 1

Setup repository.

FASE 2

Setup Supabase.

FASE 3

Database schema.

FASE 4

Frontend registrazione.

FASE 5

Generazione QR.

FASE 6

Invio email.

FASE 7

Scanner check-in.

FASE 8

Pannello admin monitor.

FASE 9

Pannello admin full.

FASE 10

Recovery QR.

FASE 11

Cleanup GDPR automatico.

21. EVOLUZIONI FUTURE

Possibili evoluzioni:

- wallet digitale;
- badge NFC;
- eventi;
- newsletter;
- notifiche push;
- gestione manifestazioni;
- statistiche avanzate;
- analytics accessi.

22. NOTE OPERATIVE IMPORTANTI

NON utilizzare:

- Google Sheets;
- TinyURL;
- codici statici;
- dati personali nel QR.

MANTENERE:

- UX semplice;
- mobile-first;
- font grandi;
- semplicità operativa.

23. OBIETTIVO FINALE

Realizzare una piattaforma:

- moderna;
- veloce;
- sicura;
- facilmente manutenibile;
- pronta per future evoluzioni;
- semplice da utilizzare per operatori e segreteria.

24. DOCUMENTI LEGACY

Il progetto attuale GAS/Google Sheets deve essere utilizzato:

- solo come riferimento funzionale;
- NON come riferimento architetturale.

La nuova implementazione deve essere completamente riscritta.

25. STATO DOCUMENTO

Questo documento rappresenta:

- specifica tecnica ufficiale;
- riferimento architetturale;

-
- guida per coding chat;
 - base progettuale della nuova piattaforma.
